

The Bitmap Datapath

video2 - every package between the framebuffer and the VGA connector,
and what two GAL22V10s would change

Derived from video2/plan.md, video/docs/graphics.md and docs/machine.md. Every package count and every placement figure in this report is **measured with hardware/place/pack.ts, the repository's own skyline packer, rather than estimated. Timing figures are derived from graphics.md**; nothing here has been on a bench.

0. What this answers, and one correction first

52 ICs is not the bitmap mode. video2/plan.md §11.2.1 measures variant E at 52 packages, and variant E is the **text-only** card - the one with bitmap mode deleted. Bitmap mode is not what costs 52; it is what the -6 lever removes. The bitmap-capable variants in that table are C at 54 and B at 58, and both of those counts came from a BOM that this report has since found to be optimistic.

So this report does three things. It **recounts the bitmap datapath package by package** against the schematics, which moves the honest figure; it gives **the reason each subsection and each part exists**, which is what the schematics are for; and it **runs the GAL question**, which turns out to be the decision that settles whether the card exists at all.

The short answer on GALs: two GAL22V10 are worth six packages, and they are the difference between a bitmap card that does not place and one that places with room to spare. They are also the most period-honest parts on the board - a 22V10 is 1986, older than the 74AHCT logic the dot path already requires.

	ICs	On a 240 mm board	Verdict
Bitmap datapath, no programmable logic	59	3 packages over, 79 % of placeable area	does not fit
The same card, 2 x GAL22V10	53	places, 73 %	this is the build
The same card, 4 x GAL22V10	50	places, 71 %	diminishing
Bitmap + text on one board, 4 GALs	58	3 over, 80 %	still does not fit
video/ today, for scale	33	places, 59 %	3 CPLDs do this work

1. The signal chain

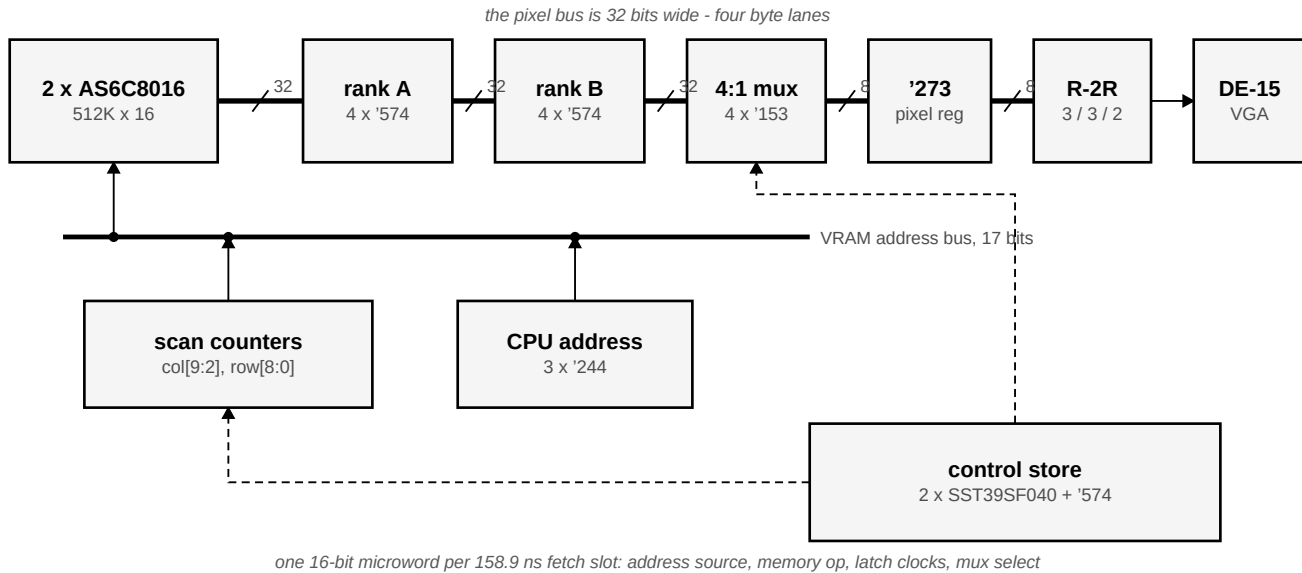


Figure 1. One byte leaves the framebuffer, crosses two rank latches and a 4:1 mux, is latched once and becomes a voltage. Everything below the datapath exists to tell it where to come from and when to move.

The whole card is one sentence: **at every dot, one byte of framebuffer must become one RGB332 voltage, and 25.175 MHz is 39.72 ns.** No memory of the period does that on its own, so the card fetches four bytes at a time into latches and serves them out over four dots. Every subsection below is a consequence of that.

- **The framebuffer array** supplies four bytes per 158.9 ns fetch slot. Two packages, because a 55 ns part cannot feed a 39.72 ns dot. (§2.1)
- **The address generators** say which four bytes. Two counters whose natural rollover is the scrolling ring, plus whatever drives them onto a shared bus. (§2.2)
- **The dot path** turns four bytes into four pixels: two ranks of latches, a 4:1 mux, one output register, three ladders. (§2.3)
- **The control plane** is the microcode: a ROM indexed by the slot counter emitting one word per slot, and a second ROM indexed by the line counter. (§2.4)
- **The host interface** puts the CPU's own address on the VRAM bus - flat memory, no pointer - and gets a byte back off a 32-bit bus. (§2.5)

2. The subsections, and why each part is there

2.1 The framebuffer array

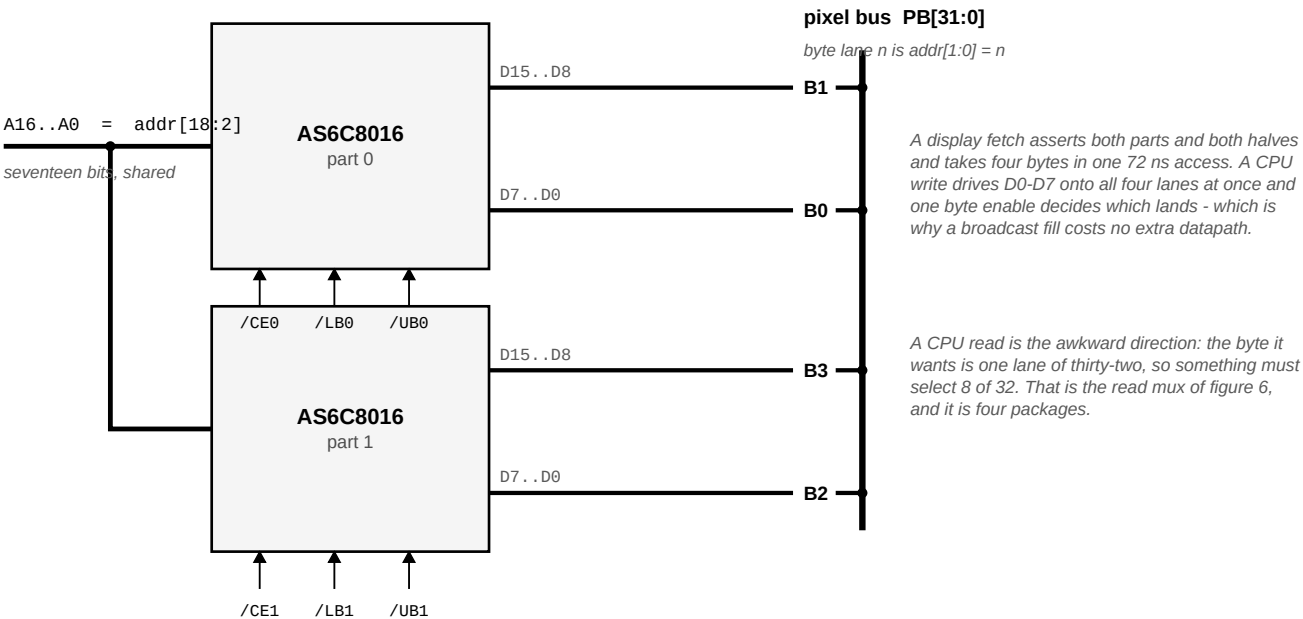


Figure 2. Two x16 parts side by side are one 32-bit word: four byte lanes, one access. The byte enables are what make a single-byte CPU access and a four-byte display fetch the same circuit.

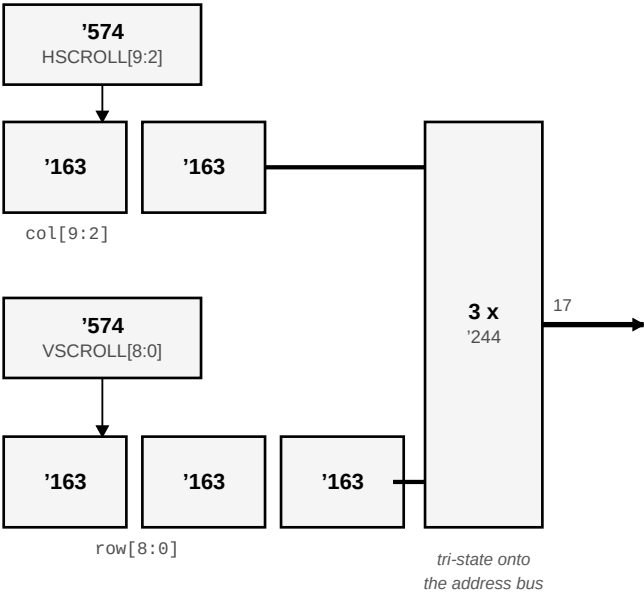
Why two packages and not one, and not four. A 640-pixel line at 8bpp is one byte per 39.72 ns dot. The AS6C8016 is a 55 ns part and Alliance list no faster grade, so a single byte-wide part is out by a factor of about four. Fetching **four bytes per 4-dot slot** turns the requirement into one 72 ns access per 158.9 ns - graphics.md §2.1's budget of 12 ns of address mux, 55 ns of SRAM and 5 ns of setup, with 86.9 ns of slack. Two x16 parts deliver those four bytes in one access; four x8 parts deliver the same four bytes and cost two more packages, and that is the period-honest 1989 build (graphics.md §15).

Part	Qty	Why it is required
AS6C8016-55, 512K x 16	2	Four bytes per fetch slot. Neither capacity nor cost drives the count - width does. 1.5 MB of the 2 MB is dead, which is graphics.md §14.2.4's own trade.
74HCT138 + 74HCT00	2	Byte-lane control: /CE, /OE, /WE, /LB and /UB on two parts is ten signals, and they are a function of the microword and addr[1:0]. This pair was missing from plan.md §11 entirely.

The one asymmetry worth knowing. A display fetch wants all four lanes and a CPU write wants one. The write direction is free: D0-D7 fan out to all four lanes and a byte enable decides which one lands - which is also why graphics.md §7.4's broadcast fill needs no datapath of its own. **The read direction is not free**, and §2.5 is where it is paid for.

2.2 Display address generation

Discrete - eight packages



Two GAL22V10 - two packages

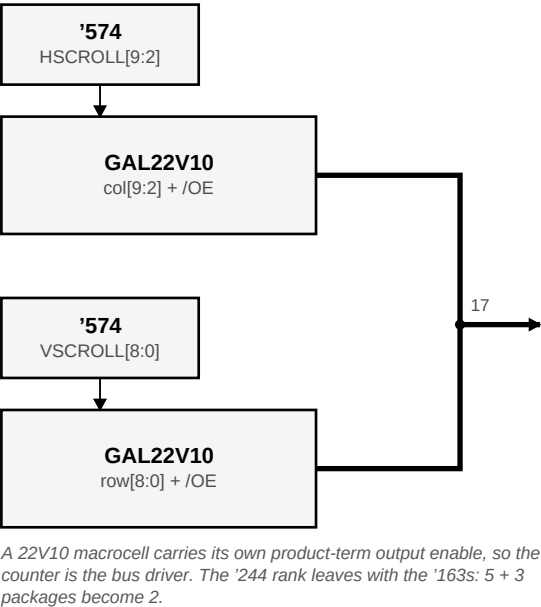


Figure 3. The same function twice: on the left the discrete build, on the right the two GALs of §4. The '244 rank on the left exists only because a '163 cannot three-state its own outputs.

Two counters, and their rollover is the feature. VRAM is a 1024 x 512 torus. The column counter holds col[9:2] - eight bits, one per 4-byte fetch group - and the row counter holds row[8:0]. Eight bits of column is 1024 pixels and nine bits of row is 512 rows, so each counter's natural rollover is exactly the scrolling ring: HSCROLL and VSCROLL wrap by construction and there is no comparator anywhere. This is graphics.md §8 transferred intact.

Part	Qty	Why it is required
74AHCT163 column counter	2	col[9:2], 8 bits, synchronously loaded from HSCROLL[9:2] at HLOAD and enabled by FETCH . SLOTTICK. Four bits per package is the whole reason this subsection is expensive.
74AHCT163 row counter	3	row[8:0], 9 bits, loaded from VSCROLL through vertical blank and stepped once per displayed line by ROWADV.
74HCT244	3	Seventeen address bits onto a shared tri-state bus. A '163 has no output enable, so the buffer is not optional - and it is exactly what a GAL deletes (§4).
74HCT574 scroll registers	3	HSCROLL is 10 bits and VSCROLL is 9. They must hold their value and present it continuously to the counters' load inputs, so they are registers and not a RAM location.

Why the counters cannot simply be wired to the SRAM. Four sources want the same seventeen address lines - the scan counters, the CPU, and in a dual-mode card the cell and font addresses. They are exclusive in time, so the cheap answer is a tri-state bus rather than a mux tree: a 4:1 mux on 17 bits is nine '153 packages, and tri-stating each source is three buffers per source. **The bus is why the '244 rank exists, and the '244 rank is why a GAL pays.**

2.3 The dot path

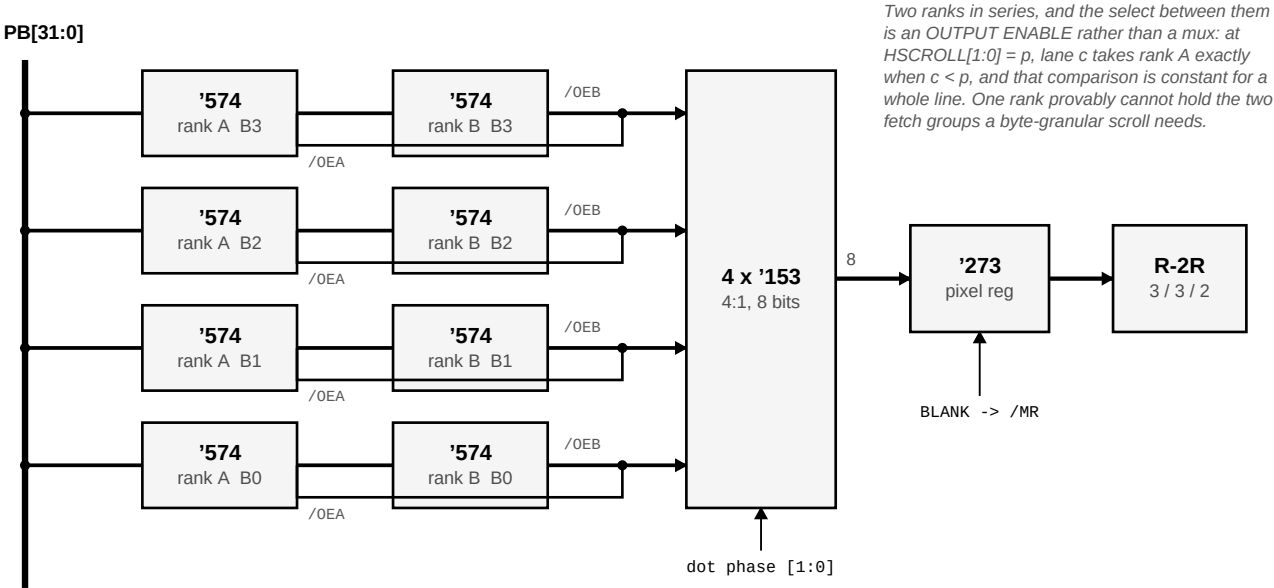


Figure 4. Two ranks of fetch latches in series, the select between them an output enable rather than a mux, then one 4:1 mux at dot rate, one register, and the ladders.

Why two ranks and not one. This is the one place where the obvious design is provably wrong, and graphics.md §8.2 pays for the proof. At $HSCROLL[1:0] = p$ the leftmost displayed pixel of slot s^* is byte $4s+p$, so the four mux inputs must carry bytes $4s+p$ through $4s+p+3$ - and those straddle **two** fetch groups. A single rank always holds the most recent fetch, whichever clock edge is chosen, so at any p other than zero the line steps backwards four minus p pixels in. Rank A holds group $s+1$, rank B holds group s , and lane c presents rank A exactly when $c < p$.

And that comparison is constant for a whole line, because HSCROLL is loaded at HLOAD and the lane index is wiring. So the choice between the ranks is an **output enable**, not a multiplexer - which is the difference between the +12 packages graphics.md §19 item 28 first estimated and the +4 it actually cost.

Part	Qty	Why it is required
74AHCT574 rank A	4	The fetch latch: 32 bits, clocked mid-slot at 144 ns so it is stable for the next slot's four pixel times.
74AHCT574 rank B	4	The previous group, so the two live groups a byte-granular scroll needs are both present. Not optional - see above.
74AHCT153 pixel mux	4	4:1 on 8 bits at dot rate: which of the four latched bytes is this dot. Budgeted as the default rather than the fallback, because graphics.md §19 item 2's tri-state alternative has never been benched.
74AHCT273 output register	1	One register, 8 bits - and its asynchronous /MR is blank-to-black . Three-stating a register floats the ladder, which is not 0 V; forcing an index is meaningless when the index is the colour (graphics.md §9.2).
74HCT32 rank select	1	/OEA and /OEB per lane from HSCROLL[1:0]. A '574 has one output enable, so the pair must be complements in silicon.

What deleting the palette LUT bought here. graphics.md §6.1 gives the chain index latch - 15 ns LUT - output latch **11.7 ns of margin in a 39.72 ns dot**, and §8 and §9 of features.md both cite that margin as the reason a hardware sprite, a hardware cursor and a border are all impossible on that card. With a fixed RGB332 ladder the chain is mux plus register: about 14 ns used and **25 ns spare**. That is the single largest timing improvement in video2, and it is a consequence of the colour decision rather than of any effort.

2.4 The control plane

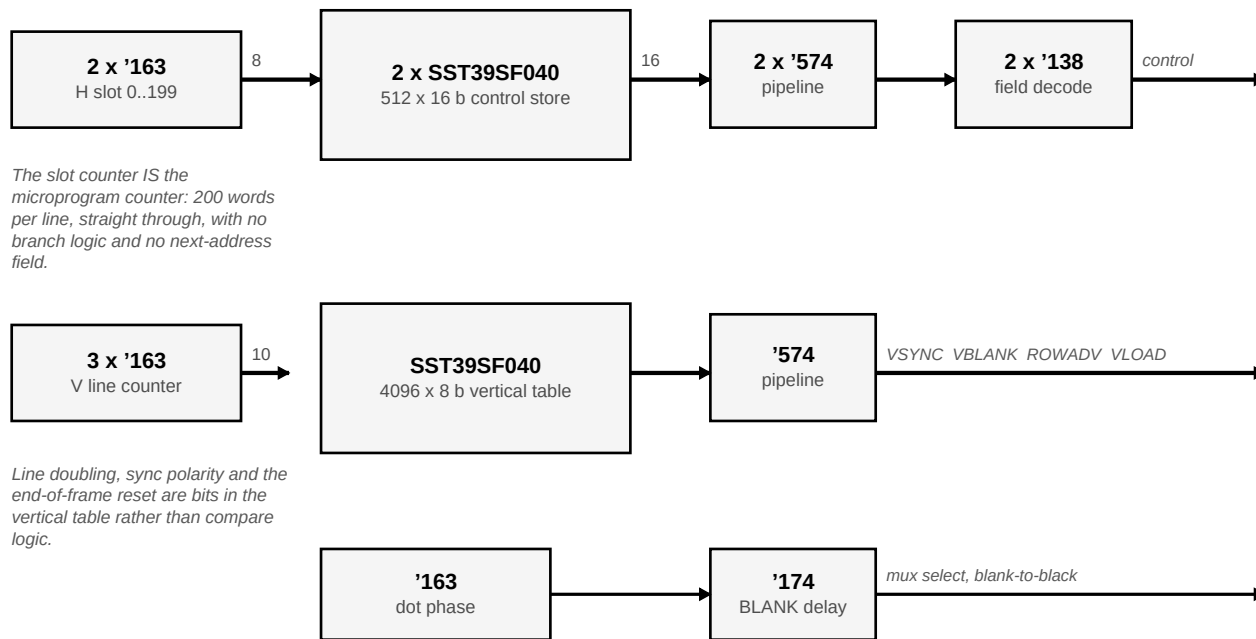
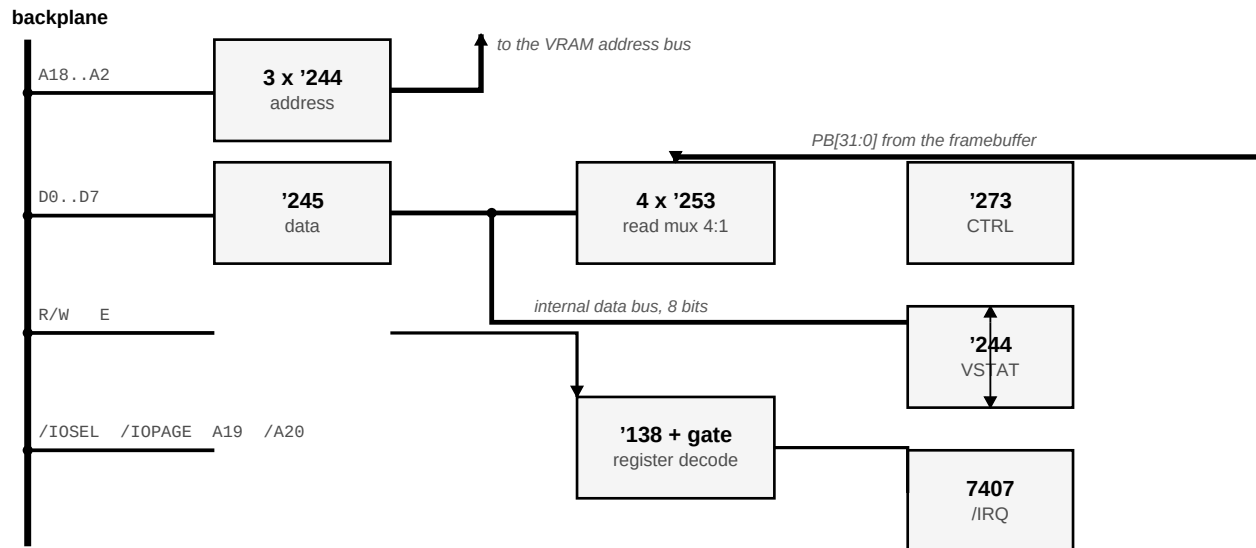


Figure 5. Two ROM tables and their pipeline registers. The horizontal store is addressed by the slot counter and the vertical table by the line counter; neither needs a sequencer, because neither ever branches.

The slot counter is the microprogram counter. A line is 200 fetch slots and the counter visits them in order, every line, for ever. A control store indexed by that counter needs no branch logic, no condition codes, no next-address field and no state register of its own: the microprogram is 200 words long and runs straight through. VMODE does not reach it at all, because all four modes are 800 dots with the same horizontal sync - only the vertical timing differs.

Part	Qty	Why it is required
SST39SF040 control store	2	A 16-bit microword per slot. Two parts because the microword is 16 bits and the part is 8. The same flash the motherboard's boot ROM uses, with a datasheet-verified pinout in hardware/lib/parts.ts. 0.1 % of it is used.
SST39SF040 vertical table	1	VSYNC already at the mode's polarity, VBLANK, ROWADV, VLOAD, end-of-frame reset. Sync polarity and line doubling become data rather than the 10 product terms and the parity argument graphics.md §6.2.1 and §8.1 spend on them.
74AHCT574 pipeline	3	The store is read one slot ahead and the register presents its word for the whole of the next: 70 ns of flash plus 5 ns of setup inside 158.9 ns, 83.9 ns of margin - the most comfortable number on the card.
74HCT138 field decode	2	Three encoded fields - address source, memory op, latch select - whose members are mutually exclusive by construction.
74AHCT163 H slot counter	2	0..199, and it is the microprogram counter. Reset comes from the store itself, so there is no comparator.
74AHCT163 V line counter	3	0..448 or 0..524. Reset is a bit in the vertical table, again with no comparator.
74AHCT163 dot phase	1	Two bits below the microcycle: the mux select and the output register's clock run at 25.175 MHz, and a control store built from 70 ns flash cannot.
74AHCT174 blank delay	1	BLANK must be delayed to match the pixel pipeline. graphics.md §19 item 35 is this exact defect: a picture five dots right of the active window, invisible to twelve testbenches because they compared the card against its own counter.

2.5 The host interface



The CPU's physical A18..A2 go straight onto the card's VRAM address bus, so VRAM is flat memory: no WPTR, no posted-write pointer and no read prefetch. The price is the read mux above, and 27 ns of margin in the cycle.

Figure 6. Flat memory: the CPU's own physical address goes onto the VRAM address bus. The read mux is the price, and it is four packages.

Flat addressing deletes more than it costs. The CPU's physical A18-A2 are on the backplane and the card has room to buffer them, so a store to the VRAM window writes the byte at that address. That removes WPTR's nineteen registers, the posted-write latch, the write-pointer reload rule software had to keep, VDATA, and the read prefetch with its invalidation rules - together, most of graphics.md's 2026-09-10 and 2026-09-11 defect list. It also means TFM works on VRAM directly: a fill at 3 bus cycles per byte, with no setup writes at all.

Part	Qty	Why it is required
74HCT244 CPU address	3	Seventeen bits onto the shared VRAM address bus, tri-stated outside the CPU's half-slot.
74HCT253 read mux	4	The one genuinely awkward package group on the card. The pixel bus is 32 bits and a CPU read wants 8 of them, so something must select one lane of four. The display's own '153 mux cannot be borrowed: its select is the dot phase, and stealing it would put a hole in the picture - which is graphics.md §19 item 38, found by a running machine. Missing from plan.md §11.
74HCT245	1	D0-D7 to the card's internal bus, direction from R/W. HCT and not HC: the CPU module drives 3.3 V.
74HCT138 + 74HCT30	2	The register decode. Seven address bits, not six - with A6 outside /IOSEL, a card matching six answers at its base and 64 bytes below it (machine.md §5 item 1 A).
74HCT273 CTRL	1	The mode register, with /MR from backplane /RESET so the card comes up with the display disabled.
74HC244 VSTAT	1	VLANK and HBLANK are live state, not a stored byte, so nothing can read them back. graphics.md §19 item 43 is what happens without this package: a driver polls a bit and reads what it last wrote, which is zero.
74HC244 sync fan-out	1	HSYNC and VSYNC to slot B24/B26. The CPU module's raster compare clocks from one and takes its frame origin from the other; without the second, the line counter has no origin and the split jitters by the whole interrupt latency.
7407	1	/IRQ is open-drain and shared. A totem-pole output on a wire-OR is a fight.

2.6 The fetch slot, and why nothing ever waits

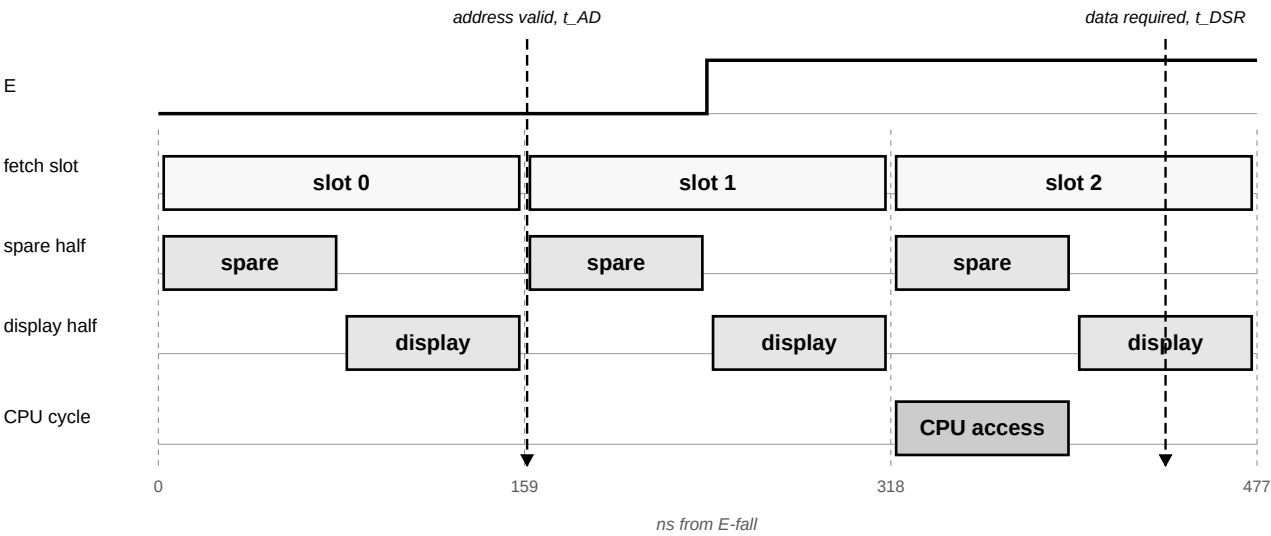


Figure 7. One E period is three fetch slots. The CPU's access takes the front half of a slot and the display fetch the back half, so the two never meet.

The card never asserts /WAIT. Each slot holds two 72 ns accesses in 158.9 ns; the display takes one and there is no span writer, no display list and no map fetch competing for the other. So the CPU's access is always available in the slot its cycle lands in, and the arbitration that cost graphics.md three defects on the day the machine first ran its own boot ROM reduces to a wire.

The spare access goes first, and that is a specification rather than a preference. graphics.md §5.2.2 shows the read budget does not close the other way round. Taking the access in the front half of slot 2 puts data back at about 410 ns against a deadline near 437: **about 27 ns of margin, at divide-by-12 only**. At divide-by-8 it does not close, which is the same conclusion graphics.md §11 reached about a flat read - and the machine is specified at divide-by-12.

3. The recount

Drawing the schematics moved the number. Two package groups in the figures above appear nowhere in p1an.md §11, and both are load-bearing:

- **+4, the CPU read mux** (§2.5). The pixel bus is 32 bits wide and a CPU read wants one lane of it.
- **+2, framebuffer byte-lane control** (§2.1). Ten control signals across two parts, from the microword and addr[1:0].

Subsection	Packages	What is in it
Framebuffer array	4	2 SRAM, 2 byte-lane control
Display address generation	11	5 counters, 3 bus buffers, 3 scroll registers
The dot path	14	8 fetch latches, 4 mux, 1 output register, 1 rank select
The control plane	15	3 flash, 3 pipeline, 2 decode, 5 counters, 1 blank delay
The host interface	14	3 address, 1 '245, 4 read mux, 2 decode, 4 register and status
Glue	1	inverters
Bitmap datapath, total	59	and the VGA drive stage, which is 3 transistors and no ICs

59 packages do not place on a 240 mm board - three over, at 79 % of placeable area, with the skyline against its limit. The dual-mode card, bitmap and text together, recounts from p1an.md's 62 to **67** and is eleven over. Neither number is an estimate: both come from hardware/place/package.ts.

4. Would one or two GAL22V10 substantially reduce this?

Yes - six packages for two, and it is the difference between a card that places and one that does not. The reasoning is not that a GAL is denser in the abstract; it is that the two biggest discrete groups on the card are the wrong shape for 74-series parts in exactly the way a 22V10 is the right shape.

4.1 Why the scan counters, and not somewhere else

A GAL22V10 is ten macrocells, each registered or combinational, **each with its own product-term output enable**, behind twelve dedicated inputs and ten I/O pins. Three properties of the display address generator line up with that exactly:

- **It is counter bits.** A '163 is four bits per package; a 22V10 is ten. `graphics.md §10.1.1` found the same thing from the other direction - that card was programmable-logic-heavy **because** about 54 of its macrocells were counter bits.
- **It needs a synchronous parallel load**, which is what the scroll registers feed, and that is ordinary registered-macrocell logic.
- **It must three-state onto a shared bus**, and this is the part that pays twice: because the output enable is per macrocell and driven by a product term, **the counter is its own bus driver and the '244 rank disappears with the '163s.**

	Discrete	2 x GAL22V10
Column counter, <code>col[9:2]</code>	2 x '163	8 registered macrocells
Row counter, <code>row[8:0]</code>	3 x '163	9 registered macrocells
Onto the address bus	3 x '244	the macrocells' own /OE
Packages	8	2

This is not a new idea; it is what video/ did. That card's archived GAL build - 41 ICs including ten GAL22V10 - spent exactly two of them on the scan address (`video/docs/history.md §10.1`), and `graphics.md §10.1.2` records the property this report is relying on: **on GALs the scan and WPTR pairs tri-state onto a shared bus for free**.

4.2 Does it fit? The product-term arithmetic

A synchronous counter bit with parallel load and count enable is $D_n = \text{LOAD} \cdot \text{In} + \text{/LOAD} \cdot (Q_n \text{ XOR } (\text{EN} \cdot Q_0 \cdot \dots \cdot Q_{n-1}))$. Expanded to sum-of-products that is **$n + 3$ product terms for bit n** - one for the load, one for the hold under /EN, one per lower bit that breaks the carry, and one for the toggle. The top bit of the 9-bit row counter therefore needs 11 terms.

A 22V10's macrocells are not uniform: they carry 8, 10, 12, 14, 16, 16, 14, 12, 10 and 8 product terms. Sorting the nine counter bits by need against that allocation gives 11 into 16, 10 into 16, 9 into 14, 8 into 14, 7 into 12, 6 into 12, 5 into 10, 4 into 10 and 3 into 8: **every bit fits, with the widest at 11 of 16**. The eight-bit column counter is looser still and leaves two macrocells spare.

Pins are tighter than product terms, and that is the risk to check. The row counter GAL needs `VSCROLL[8:0]`, `ROWADV`, `VLOAD` and an output-enable term - twelve inputs - with nine outputs, against twelve dedicated inputs plus the one I/O pin left over. **It fits with about one pin to spare, and the fit must be run rather than asserted.** The repository already has the toolchain for that: `hardware/gal/jedec/` assembles the fuse map and `check:cupl` cross-checks it against Atmel's own compiler.

4.3 What it measures

Variant	ICs	240 mm	Area
Bitmap datapath, no programmable logic	59	3 over	79 %
+ 2 x GAL22V10 on the scan counters	53	places	73 %
+ 4 GALs (scan, plus decoders and glue)	50	places	71 %
2 GALs, and read-back given up	49	places	68 %
2 GALs, and the tri-state pixel bus closes	49	places	68 %
2 GALs, wide microword instead of 2 x '138	53	places	75 %

Variant	ICs	240 mm	Area
Bitmap + text, no programmable logic	67	11 over	88 %
Bitmap + text, 2 GALs	61	5 over	82 %
Bitmap + text, 4 GALs	58	3 over	80 %

Three things fall out of that table and only the first was expected.

- **Two GALs are enough, and four are not needed.** The first two delete four packages each; the next two delete about one and a half each, because what is left is decoders and gate packs rather than counter bits. **Two is the knee.**
- **Do not buy the placement by deleting a feature.** Giving up VRAM read-back saves four packages and the card already places without that sacrifice. The same goes for the tri-state pixel bus, which is an unbentched hope (graphics.md §19 item 2) - with two GALs it becomes **margin** rather than a dependency, which is the right place for it.
- **The dual-mode card still does not fit, even with four GALs.** Bitmap and text on one 240 mm board is three packages over at 58. Closing that needs either more GALs than this question contemplates, or a two-board set, or one of plan.md §11.3's levers. **If the card is to be dual-mode, this is the decision that has to be taken first.**

One measured non-result, recorded because it looked promising: **widening the microword to 24 bits and deleting the two '138 field decoders is IC-neutral - a third flash and a third pipeline register cost exactly what the decoders save - and it is worse on area, 75 % against 73 %, because a DIP-32 flash is a much larger package than a DIP-16 decoder. It would buy a little propagation margin and nothing else.**

4.4 What two GALs cost

Period	A GAL22V10 is 1986 . It is older than the 74AHCT logic the dot path requires (~1990) and far older than the framebuffer (~1995). On the period axis this makes the card more honest, not less.
The house rule	video2's brief says no CPLDs and no GALs, and that is a style rule rather than a period one - the root README.md retired the machine-wide version of it on exactly this argument. Two GALs on one card is the owner's call, and it is the whole subject of this section.
Verification	Cheaper than the alternative. The repository already assembles GAL fuse maps and checks them against Atmel's CUPL over every input combination. The ROM tables need a new checker either way.
What is given up	Two functions stop being visible on a scope pin by pin, which is the real content of the style rule. The counters' outputs are still bus signals and still observable; what disappears is the '244 rank and the carry chain between packages.
The fit	Unrun. Product terms are comfortable (§4.2); pins are not , at roughly twelve of thirteen on the row counter. This is the one thing that could send the answer back.

4.5 Recommendation

Take the two GALs, on the column and row counters, and spend them on placement rather than on features.

That gives a bitmap card of **53 packages that places on a 240 mm board at 73 % of placeable area**, with VRAM read-back intact and with graphics.md §19 item 2's unbentched tri-state pixel bus held back as four packages of margin rather than relied upon.

Do not take four. The third and fourth GALs absorb decoders and gate packs at about 1.5 packages each, they put more of the card behind a fuse map for less return, and the card already fits without them.

And decide the dual-mode question separately and first, because no number of GALs in this range fixes it: bitmap and text together is 58 packages with four GALs and the board holds about 55 of this mix.

5. Open items this report raises

- **Fit both GALs before anything else.** §4.2's product-term arithmetic is sound and its pin budget is not comfortable. hardware/gal/ has the toolchain; a refused fit changes the recommendation.
- **Bench the CPU read.** §2.6's ~27 ns is derived from graphics.md's own figures and assumes a fixed alignment between E and the fetch slot. Measure the access's position inside the slot directly - graphics.md §19 item 19 says not to infer it from a working read, because a marginal read works until it does not.

- **Re-run the recount into** hardware/place/parts.ts. **The 59 and 53 here come from a scratch BOM driving the repository's packer. Until there is a video2 entry in the parts list and a board file with an icBudget, check:place cannot assert any of it.**
- **The read mux deserves one more look.** Four packages to move 8 bits of 32 is the worst ratio on the card. A pair of GALs would do it in two but cannot - 32 data inputs plus a select exceeds a 22V10's 22 - and splitting it four bits per GAL is two packages for two, which is no better than the '253s.
- **Nothing here has been simulated.** graphics.md's three worst defects were all found by executing an instruction, not by reading a schematic.